Title of the Paper

Motoki Nakajima
St. Paul's School
motoki.nakajima@sps.edu

March 3, 2026

1 Abstract

Seating arrangements around a Harkness table can significantly influence the quality of discussions among students. This paper presents a mathematical model to quantify the quality of discussions based on student affinities and seating positions, and proposes algorithms to optimize the seating arrangement for maximum discussion quality. We compare a greedy algorithm and a beam search algorithm against a brute force approach to evaluate their effectiveness in finding optimal arrangements. Our results indicate that the beam search algorithm consistently outperforms the greedy algorithm, although it does not always reach the optimal solution provided by the brute force method, especially as the number of students increases. The findings suggest that while the beam search algorithm is a practical solution for real-world scenarios, there is still room for improvement in the search strategy or heuristics used to further enhance the quality of discussions around the Harkness table.

2 Introduction

Harkness table discussions, adopted by St. Paul's School's Humanities Department, aim to foster student engagement and critical thinking through collaborative dialogue. Although the table geometry naturally encourages student interaction, case specific incompatibilities may arise due to varying student personalities and discussion dynamics. This paper focuses on the seating arrangement of a Harkness table to maximize the overall quality of discussions, which we quantify using an experience-based math model then solve with scientific computations.

3 Related Background

Mrs. Buckles, a Humanities teacher at St. Paul's School, has observed discussion dynamics and identified key factors influencing the quality of Harkness discussions. She pointed out that cross-talks between students directly affects the quality of discussions. Cross-talks often occur between students who already have established relationships, which can help engage others but can also lead to exclusion of the less connected. Mrs. Buckles suggested three rules to help arrange students effectively:

- Rule 1: Two students with strong affinity should not sit next to each other.
- Rule 2: Two students with strong affinity could sit across the table but it is not ideal.
- Rule 3: Two students with strong affinity should sit facing diagonally across the table.

4 Formal Problem Definition

The problem models an elliptical harkness table discussion geometrically. The score of each n student's contribution to the table is defined by the student's affinity level with the other students, while considering a geometric cross-talk coefficient that depends on the angle between the students.

$$S_i = a_{ij} \cdot k_{ij} \quad \text{(for all } j \neq i\text{)}$$

$$k_{ij} = d_{ij} \cdot \left(\sin^{16}(\theta_{ij}) + 0.3\cos^{8}(\theta_{ij}) - 4.0\cos^{64}(\tfrac{\theta_{ij}}{2})\right)$$

S_i is the score of student i, a_{ij} is the affinity level between student i and student j, k_{ij} is the cross-talk coefficient between student i and student j, θ_{ij} is the central angle between student i and student j, and d_{ij} is the distance between student i and student j. a_{ij} is normalized to be between 0 and 1, where 0 indicates no affinity and 1 indicates strong affinity. d_{ij} uses the Euclidean distance where the minor axis of the table is 1 and the major axis is $\frac{11}{7} \approx 1.57$. The aspect ratio is based on the actual table used in St. Paul's School. The total score of the table arrangement is defined as the sum of all scores among the students.

$$S = \sum_{i=1}^{n} S_i$$

The geometrically determined cross-talk coefficient k_{ij} is modeled based on feedback from Humanities teachers at St. Paul's School, and it is designed to capture the complex relationship between the angle of seating and the level of cross-talk.

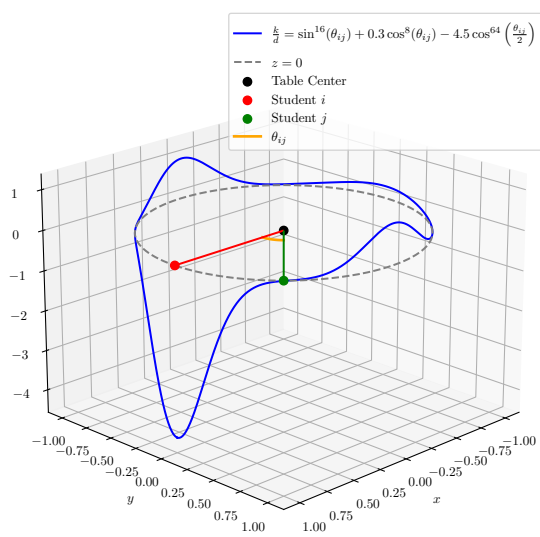

Figure 1: Cross-talk coefficient $\frac{k_{ij}}{d_{ij}}$ around the table.

The goal of the problem is to find the arrangement of students around the elliptical table that maximizes the total score S.

5 Algorithm Description

5.1 Greedy Algorithm

The greedy algorithm iteratively searches through the best possible seating for each student based on the expected gain in total score S. Starting from an empty table, the algorithm picks a random student who has not been seated yet and calculates the expected gain in total score S for each possible seating position around the table. The student is then seated in the position that yields the highest expected gain in total score S. This process is repeated until all students are seated.

5.2 BeamSearch Algorithm

The BeamSearch algorithm heuristically explores the search space of possible seating arrangements by maintaining B of best arrangements at each step, where B is the beam width. Starting from an empty table, the algorithm generates all possible seating arrangements for the first student and evaluates their total scores S. The top B arrangements are kept for the next iteration, where the next student is seated in all possible positions for each of the B arrangements. The total scores S of the new arrangements are evaluated, and the top B arrangements are kept for the next iteration. This process continues until all students are seated, and the arrangement with the highest total score S is returned as the solution.

The algorithm implemented in this paper uses two additional heuristics to further optimize the search process:

- Heuristic 1: For each step, the algorithm maintains the greedy option as one of the candidates, ensuring that the best local choice is always considered.
- Heuristic 2: For each step, the algorithm limits the number of candidates branched from the same parent arrangement to 2, which helps to diversify the search and avoid getting stuck in local optima.

5.3 Brute Force Algorithm

The brute force algorithm exhaustively enumerates all possible seating arrangements of the students around the table and calculates the total score S for each arrangement. This algorithm was only used to verify the correctness of the other algorithms for small numbers of students due to its factorial time complexity.

6 Correctness Argument

6.1 Greedy Algorithm

The greedy algorithm does not guarantee an optimal solution because it makes locally optimal choices at each step without considering the global arrangement of students.

6.2 BeamSearch Algorithm

The BeamSearch does not guarantee an optimal solution because it limits the search space, which may lead to missing optimal arrangements. However, heuristics are implemented, which can help to find better arrangements than the greedy algorithm. Considering the BeamSearch algorithm always includes the greedy solution as a candidate, the outcome of this algorithm is guaranteed to be at least as good as the greedy algorithm.

6.3 Brute Force Algorithm

Because the brute force algorithm searches through all possible arrangements, the optimal arrangement is guaranteed.

7 Complexity Analysis

7.1 Greedy Algorithm

Each step scans all valid $(student, seat)$, where each scan calls $O(1)$ to calculate the expected gain.

$$T_{\text{greedy}}(n) = O\left(\sum_{r=1}^{n} r^2\right) = O(n^3).$$

7.2 BeamSearch Algorithm

Each step scans $O(B)$ arrangements, where each arrangement has to explore all valid $(student, seat)$, which creates $O(Bn^2)$ candidates.

The dominant factor of the time complexity resides in the sorting among the candidates.

$$T_{\text{beam}}(n) = O\left(\sum_{i=1}^{n} B \cdot r^2 \cdot \log(Br)\right) = O\left(Bn^3 \log(Bn)\right).$$

7.3 Brute Force Algorithm

The brute force algorithm explores all possible arrangements, which is $n!$.

$$T_{\text{brute}}(n) = O(n!).$$

8 Experimental Results

8.1 Greedy Algorithm vs. BeamSearch Algorithm

For the greedy algorithm and the beam search algorithm, we observed improvements in the total score S as the number of beams increased, with the beam search algorithm consistently outperforming the greedy algorithm.

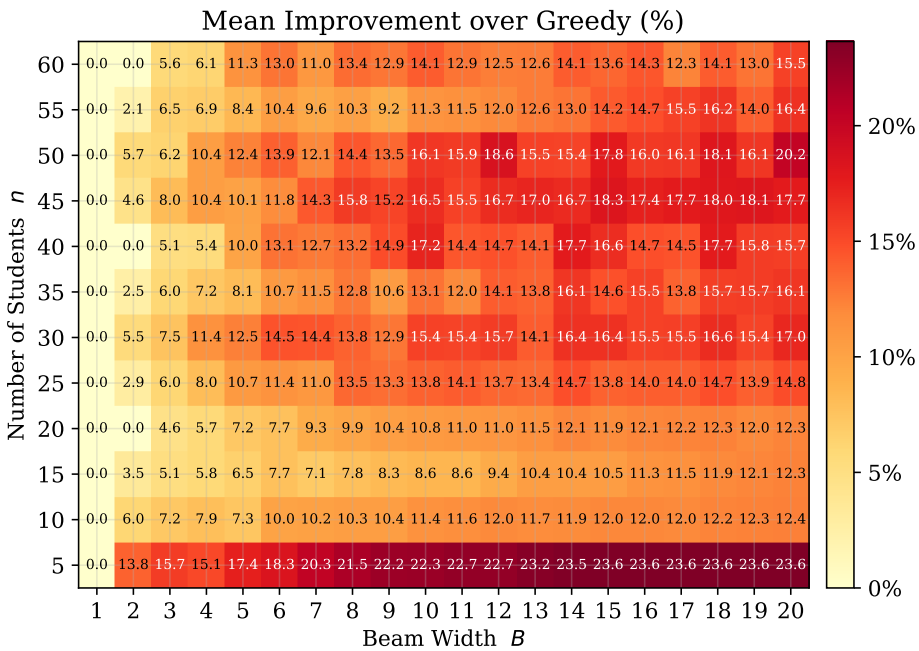

Figure 2: Improvement of the total score S over the greedy algorithm for different beam widths.

For each number of students n, we observed that the beam width had a significant impact on the total score S, with larger beam widths generally leading to better arrangements.

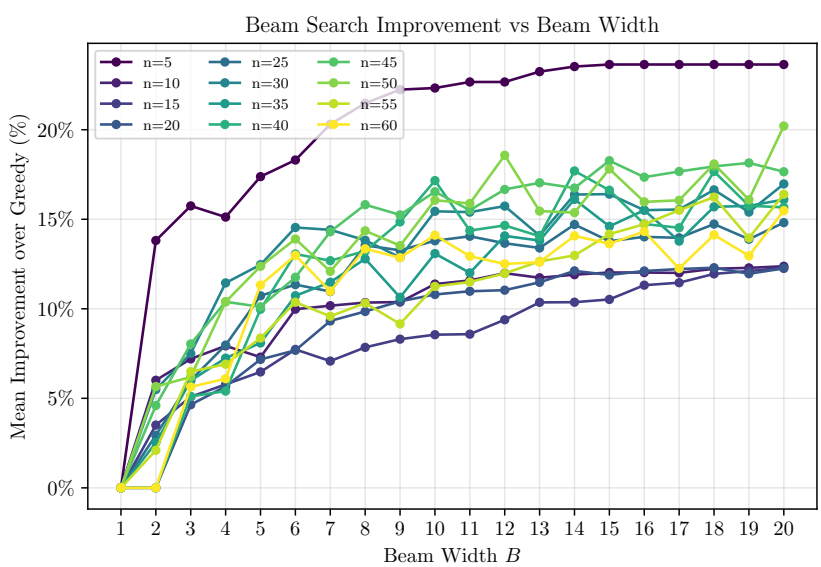

Figure 3: Total score S for different beam widths B across various numbers of students n.

8.2 Search Algorithms vs. Brute Force Algorithm

For small numbers of students (up to $n = 10$), we compared the total scores S obtained from the greedy algorithm and the beam search algorithm with the optimal scores obtained from the brute force algorithm. We used $B = 20$ for the beam search algorithm in this comparison.

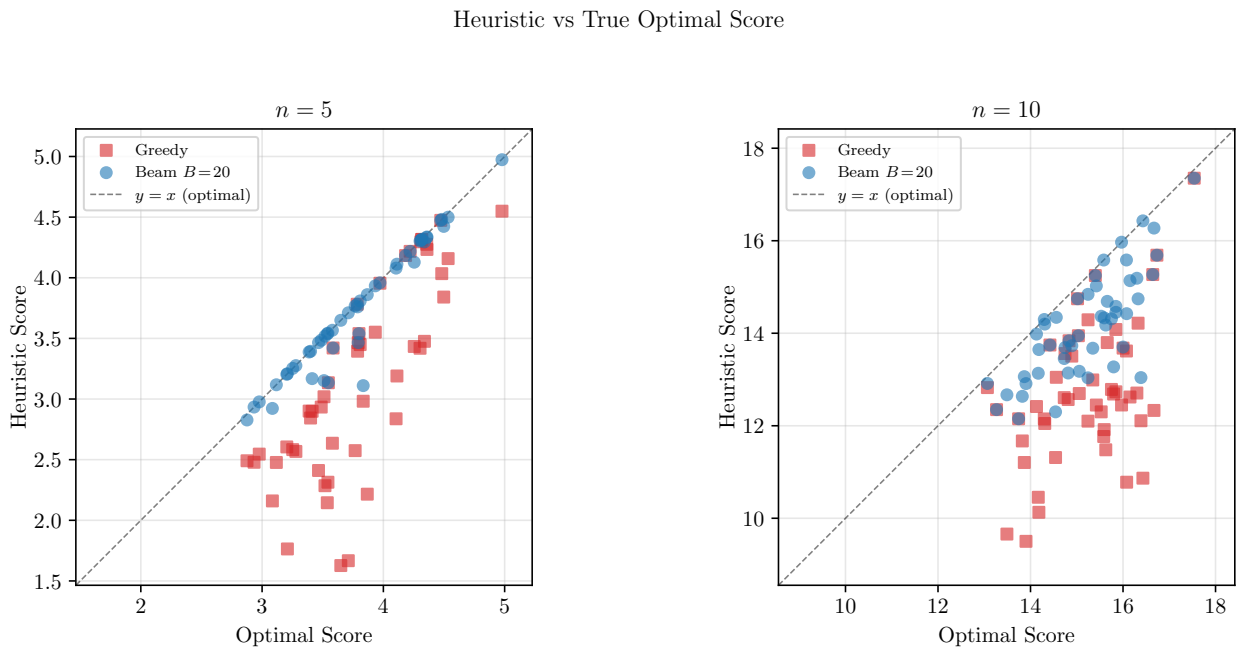

Figure 4: Comparison of total scores S from the greedy algorithm, beam search algorithm where $y = x$ represents the optimal score

Even though the beam search algorithm plateaued at $B = 20$ when $n = 5$, which could be observed in Figure 3, not all the beam search results reached the optimal score.

9 Discussion and Interpretation

We observed that the beam search algorithm consistently outperformed the greedy algorithm, with larger beam widths leading to better arrangements and higher total scores S. However, the beam search algorithm did not always reach the optimal score obtained from the brute force algorithm, especially as the number of students increased, which suggests that there is still room for improvement in the search strategy or heuristics used in the beam search algorithm. The beam search algorithm's performance between different number of students n showed small correlation when compared to the greedy algorithm, which suggests that the beam search algorithm is more effective in finding better arrangements regardless of the number of students. Considering the arrangement of students only happens once in a while, the search algorithms being transformative, and also that the student number remains relatively small, the beam search algorithm with a large beam width can be a practical solution for finding good arrangements in real-world scenarios.

10 Aknowledgements

Mrs. Buckles, a Humanities teacher at St. Paul's School, provided insights and feedback that were instrumental in modeling the problem and designing the algorithms.

Gemini, an AI language model developed by Google DeepMind, was used to assist in the development of the algorithms.